

# CONTAINER\_ARCHITECTURE

## Helix OTA — Container Architecture

### Overview

This diagram shows the **Docker container architecture** for the Helix OTA platform, including all services, their inter-container networking, volume mounts, exposed ports, and health checks. The system is deployed via Docker Compose (dev/staging) or Kubernetes (production).

---

### Diagram

flowchart TB

```
subgraph Internet["🌐 Internet"]
  OPERATOR["👤 Operator<br/>(Browser)"]
  DEVICES["📱 Device Fleet<br/>(Android Clients)"]
end
```

```
subgraph DockerNetwork["🐳 Docker Network: helix-ota-net (bridge)"]
```

```
  subgraph ReverseProxy["🔄 Reverse Proxy Layer"]
    NGINX["🟢 nginx<br/>-----<br/>Image: nginx:1.25-alpine<br/>Ports: 80→443 redirect,<br/>443→upstream<br/>Volumes: ./nginx/ssl,<br/>./nginx/conf.d<br/>Health: GET /healthz<br/>-----<br/>• TLS 1.3 termination<br/>• HSTS headers<br/>• Rate limiting<br/>• Static file serving<br/>• WebSocket proxy"]
  end
```

```
  subgraph AppServices["⚡ Application Services"]
    API["🟡 helix-ota-api<br/>-----<br/>Image: helix-ota-api:latest<br/>Port: 8080 (internal)<br/>>Env: .env.api<br/>Volumes: /tmp/uploads<br/>Health: GET /api/v1/health<br/>Replicas: 2<br/>-----<br/>• Go + Gin REST API<br/>• Device management<br/>• Campaign orchestration<br/>• Artifact serving"]
    SCHEDULER["🟡 helix-ota-scheduler<br/>-----<br/>Image: helix-ota-scheduler:latest<br/>Port:
```

```

8081 (metrics)<br/>Env: .env.scheduler<br/>Health: GET /
healthz<br/>Replicas: 1 (leader election)<br/>
>—————<br/>• Rollout stage evaluation<br/>• Device
selection<br/>• Auto-pause/halt logic<br/>• Campaign lifecycle"]
    WORKER["🟡 helix-ota-worker<br/>—————<br/>
>Image: helix-ota-worker:latest<br/>Port: 8082 (metrics)<br/>
>Env: .env.worker<br/>Health: GET /healthz<br/>Replicas: 2<br/>
>—————<br/>• Artifact validation<br/>• Hash
computation<br/>• Signature verification<br/>• Malware
scanning<br/>• Report aggregation"]
    DASHBOARD["🟣 helix-ota-dashboard<br/>
>—————<br/>Image: helix-ota-dashboard:latest<br/>Port:
3000 (internal)<br/>Env: .env.dashboard<br/>Health: GET /api/
health<br/>Replicas: 1<br/>—————<br/>• Next.js SSR<br/>
>• React SPA<br/>• WebSocket client<br/>• Real-time updates"]
end

```

```

    subgraph DataStores["🗄️ Data Stores"]
        PG["🐘 postgres<br/>—————<br/>Image:
postgres:16-alpine<br/>Port: 5432 (internal)<br/>Volumes:
pg_data:/var/lib/<br/>postgresql/data<br/>Env: POSTGRES_DB, USER,
PASS<br/>Health: pg_isready<br/>—————<br/>• Primary
data store<br/>• Devices, campaigns<br/>• Reports, audit log<br/>•
Connection pool: 100"]
        REDIS["⚡ redis<br/>—————<br/>Image:
redis:7-alpine<br/>Port: 6379 (internal)<br/>Volumes: redis_data:/
data<br/>Command: redis-server<br/> --maxmemory 512mb<br/> --
maxmemory-policy allkeys-lru<br/>Health: redis-cli ping<br/>
>—————<br/>• Device heartbeat cache<br/>• Session
store<br/>• Rate limit counters<br/>• Rollout state cache"]
        MINIO["📦 minio<br/>—————<br/>Image:
minio/minio:latest<br/>Ports: 9000 (API),<br/>9001 (Console)<br/>
>Volumes: minio_data:/data<br/>Env: MINIO_ROOT_USER,<br/>
>MINIO_ROOT_PASSWORD<br/>Command: server /data<br/> --console-
address :9001<br/>Health: GET /minio/<br/>health/live<br/>
>—————<br/>• Artifact storage<br/>• Presigned URLs<br/>
>• Quarantine bucket<br/>• SSE encryption"]
    end

```

```

    subgraph Observability["📊 Observability Stack"]
        PROM["📈 prometheus<br/>—————<br/>Image:
prom/prometheus:latest<br/>Port: 9090 (internal)<br/>Volumes: ./
prometheus.yml,<br/>prom_data:/prometheus<br/>
>—————<br/>• Metrics scraping<br/>• Alert rules<br/>•
30-day retention"]
        GRAFANA["📊 grafana<br/>—————<br/>Image:
grafana/grafana:latest<br/>Port: 3001 (internal)<br/>Volumes:
grafana_data:<br/>/var/lib/grafana<br/>—————<br/>•
Dashboards<br/>• Alerts visualization<br/>• API metrics<br/>•
Rollout progress"]
        LOKI["🔍 loki<br/>—————<br/>Image:
grafana/loki:latest<br/>Port: 3100 (internal)<br/>Volumes:
loki_data:/loki<br/>—————<br/>• Log aggregation<br/>•
Structured log search<br/>• 14-day retention"]
    end

```

```

end
end

OPERATOR -->|"HTTPS :443"| NGINX
DEVICES -->|"HTTPS :443"| NGINX

NGINX -->|"proxy_pass /api"| API
NGINX -->|"proxy_pass /dashboard"| DASHBOARD
NGINX -->|"proxy_pass /ws"| API
NGINX -->|"proxy_pass /artifacts"| MINIO

API -->|"SQL :5432"| PG
API -->|"GET/SET :6379"| REDIS
API -->|"PUT/GET :9000"| MINIO
SCHEDULER -->|"SQL :5432"| PG
SCHEDULER -->|"GET/SET :6379"| REDIS
WORKER -->|"SQL :5432"| PG
WORKER -->|"PUT/GET :9000"| MINIO

API -->|"metrics :9090"| PROM
SCHEDULER -->|"metrics :9090"| PROM
WORKER -->|"metrics :9090"| PROM
PROM -->|"datasource"| GRAFANA
LOKI -->|"datasource"| GRAFANA

style NGINX fill:#43A047,stroke:#1B5E20,color:#fff
style API fill:#1E88E5,stroke:#0D47A1,color:#fff
style SCHEDULER fill:#FDD835,stroke:#F57F17,color:#000
style WORKER fill:#FF8F00,stroke:#E65100,color:#fff
style DASHBOARD fill:#8E24AA,stroke:#4A148C,color:#fff
style PG fill:#42A5F5,stroke:#1565C0,color:#fff
style REDIS fill:#EF5350,stroke:#C62828,color:#fff
style MINIO fill:#FFA726,stroke:#E65100,color:#000
style PROM fill:#E53935,stroke:#B71C1C,color:#fff
style GRAFANA fill:#FF8F00,stroke:#E65100,color:#fff
style LOKI fill:#546E7A,stroke:#263238,color:#fff

```

## Container Specifications

| Service                    | Image                      | Replicas | Memory Limit | CPU Limit | Health Check                 |
|----------------------------|----------------------------|----------|--------------|-----------|------------------------------|
| <b>nginx</b>               | nginx:1.25-alpine          | 1        | 256MB        | 0.5       | GET /healthz every 10s       |
| <b>helix-ota-api</b>       | helix-ota-api:latest       | 2        | 1GB          | 1.0       | GET /api/v1/health every 15s |
| <b>helix-ota-scheduler</b> | helix-ota-scheduler:latest | 1        | 512MB        | 0.5       | GET /healthz every 15s       |

| Service                    | Image                      | Replicas | Memory Limit | CPU Limit | Health Check                     |
|----------------------------|----------------------------|----------|--------------|-----------|----------------------------------|
| <b>helix-ota-worker</b>    | helix-ota-worker:latest    | 2        | 1GB          | 1.0       | GET /healthz every 15s           |
| <b>helix-ota-dashboard</b> | helix-ota-dashboard:latest | 1        | 512MB        | 0.5       | GET /api/health every 30s        |
| <b>postgres</b>            | postgres:16-alpine         | 1        | 2GB          | 2.0       | pg_isready every 10s             |
| <b>redis</b>               | redis:7-alpine             | 1        | 512MB        | 0.5       | redis-cli ping every 10s         |
| <b>minio</b>               | minio/minio:latest         | 1        | 1GB          | 1.0       | GET /minio/health/live every 10s |
| <b>prometheus</b>          | prom/prometheus:latest     | 1        | 1GB          | 0.5       | GET /-/healthy every 30s         |
| <b>grafana</b>             | grafana/grafana:latest     | 1        | 512MB        | 0.5       | GET /api/health every 30s        |
| <b>loki</b>                | grafana/loki:latest        | 1        | 512MB        | 0.5       | GET /ready every 30s             |

## Network Configuration

| Network                   | Driver | Purpose                        | Services                                       |
|---------------------------|--------|--------------------------------|------------------------------------------------|
| <b>helix-ota-net</b>      | bridge | Internal service communication | All services                                   |
| <b>helix-ota-frontend</b> | bridge | Public-facing services only    | nginx, API, dashboard                          |
| <b>helix-ota-data</b>     | bridge | Data store isolation           | postgres, redis, minio, API, worker, scheduler |

## Volume Mounts

| Volume            | Type         | Mount Point              | Purpose                    |
|-------------------|--------------|--------------------------|----------------------------|
| <b>pg_data</b>    | named volume | /var/lib/postgresql/data | PostgreSQL persistent data |
| <b>redis_data</b> | named volume | /data                    | Redis persistence (AOF)    |

| Volume                  | Type         | Mount Point                    | Purpose                     |
|-------------------------|--------------|--------------------------------|-----------------------------|
| <b>minio_data</b>       | named volume | /data                          | MinIO object storage        |
| <b>prom_data</b>        | named volume | /prometheus                    | Prometheus metrics storage  |
| <b>grafana_data</b>     | named volume | /var/lib/grafana               | Grafana dashboards & config |
| <b>loki_data</b>        | named volume | /loki                          | Loki log storage            |
| <b>./nginx/ssl</b>      | bind mount   | /etc/nginx/ssl                 | TLS certificates            |
| <b>./nginx/conf.d</b>   | bind mount   | /etc/nginx/conf.d              | Nginx configuration         |
| <b>./prometheus.yml</b> | bind mount   | /etc/prometheus/prometheus.yml | Prometheus scrape config    |

## Docker Compose Quick Start

```
# docker-compose.yml (simplified)
version: "3.8"
services:
  nginx:
    image: nginx:1.25-alpine
    ports: ["443:443", "80:80"]
    depends_on:
      api: { condition: service_healthy }
      dashboard: { condition: service_healthy }

  api:
    image: helix-ota-api:latest
    environment:
      - DATABASE_URL=postgres://helix:secret@postgres:5432/helix_ota
      - REDIS_URL=redis://redis:6379
      - MINIO_ENDPOINT=minio:9000
    depends_on:
      postgres: { condition: service_healthy }
      redis: { condition: service_healthy }
      minio: { condition: service_healthy }

  scheduler:
    image: helix-ota-scheduler:latest
    depends_on:
      postgres: { condition: service_healthy }
      redis: { condition: service_healthy }

  worker:
    image: helix-ota-worker:latest
    depends_on:
      postgres: { condition: service_healthy }
```

```
    minio: { condition: service_healthy }

dashboard:
  image: helix-ota-dashboard:latest
  depends_on:
    api: { condition: service_healthy }

postgres:
  image: postgres:16-alpine
  volumes: ["pg_data:/var/lib/postgresql/data"]

redis:
  image: redis:7-alpine
  volumes: ["redis_data:/data"]

minio:
  image: minio/minio:latest
  command: server /data --console-address :9001
  volumes: ["minio_data:/data"]
```